

Github: <https://github.com/Fijimayowa/MAD.git>

1)

Before writing any code, I sketched the face and labeled the measurements I needed:

Face center: the middle of the canvas, $\text{Offset}(\text{size.width} / 2, \text{size.height} / 2)$.

Radius: $\text{size.shortestSide} * 0.4$, so the face always fits inside the canvas with some margin.

Mouth Rect: centered just below the face center, with a width of about radius and a height of about $\text{radius} * 0.7$.

Start angle: $0.15 * \pi$.

Sweep angle: $0.7 * \pi$.

Flutter measures angles from the 3 o'clock position and goes clockwise. That means angles between 0 and π sit on the bottom half of the rect, which is where a smile belongs. The arc is balanced because the start angle plus half the sweep is $0.15\pi + 0.35\pi = 0.5\pi$. That points straight down, so the left and right ends of the smile are symmetric around the center of the face.

2)

Every position and size in my painter comes from size or radius:

The face center comes from size.width and size.height .

The radius comes from size.shortestSide .

The mouth rect's position, width, and height are all multiples of radius.

Even the stroke width of the mouth is $\text{radius} * 0.07$.

I did not use any fixed pixel values. This matters because phones have different screen sizes and densities. If I had written something like `width: 120`, the mouth would look fine on one phone and too big or too small on another. Because everything is a proportion of radius, the whole face scales together and keeps the same shape.

3) I ran the app on the phone emulator in both portrait and landscape and saved a screenshot of each.

The change I made: at first I based the radius on `size.width`. In portrait that looked fine, but in landscape the face was much too big and got cut off at the top and bottom, because the screen is wider than it is tall. I switched to `size.shortestSide`, so the face is always sized to the smaller dimension and fits in either orientation. Since the mouth and stroke width are calculated from radius, they shrank with the face and stayed in proportion. After the change, the smile stayed centered and fully visible in both orientations.

4)Flutter calls `shouldRepaint` when the painter is replaced with a new one, to decide whether the canvas needs to be redrawn. My painter's drawing depends on the mood, so:

When the mood changes, it should return true. The picture is now different, so if it returned false, Flutter would keep showing the old face and the screen would be wrong.

When the inputs stay the same, it should return false. The new painter would draw exactly the same picture, so redrawing wastes work. Skipping it saves frame time and battery.

I could make it always return true, and the face would still look right, but it would repaint for no reason. By comparing only the things that affect the drawing (`old.mood != mood`), the painter is both correct and efficient.